

Lap Pham

CSC 413 - 01 SOFTWARE DEVELOPMENT Fall 2024

Github: <https://github.com/csc413-SFSU-FA2024/tankgame-lap1597/tree/main>

Table of contents

Introductions.....	2
1. Project Overview:.....	2
2. Introduction of the Game:.....	2
Development environment.....	2
How to build.....	3
1. Importing project.....	3
2. How to build JAR.....	7
3. How to run the Game.....	11
4. Game rules:.....	13
5. Game Assumptions:.....	13
6. Diagram:.....	14
7. Class Descriptions:.....	14
8. Reflection:.....	16
9. Project Conclusion:.....	16

Introductions

1. Project Overview:

The term project for CSC 413 involved building 2D games in Java. This project aimed to practice object-oriented programming (OOP) and source code reusability in the process of developing a 2D game. Utilizing the starter code provided by Professor Anthony Souze, I began implementing the game and applied the concepts throughout the semester in this project.

2. Introduction of the Game:

In my game, I used the concept of a Tank Game but replaced the tanks with Dog and Cat sprites. The maps are designed to simulate a farm setting where the dog and cat engage in battles against each other.

Development environment

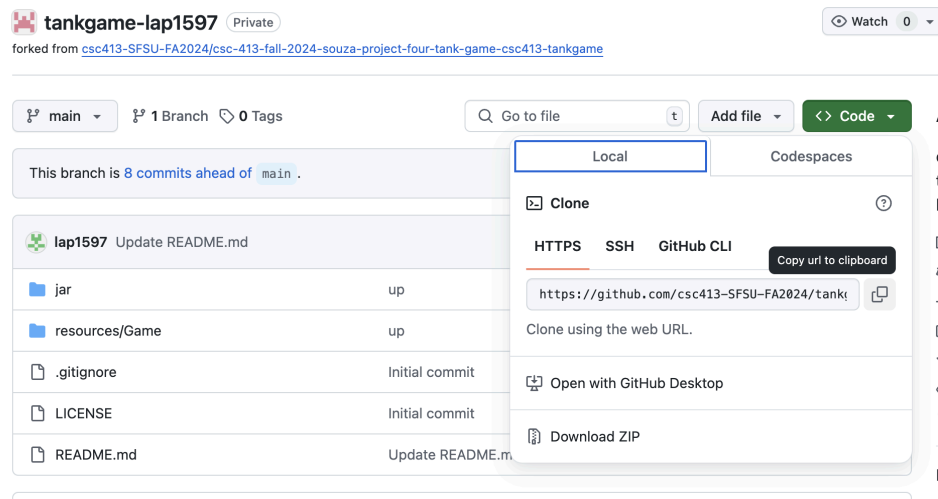
1. The version of Java Used: Java 22 - 22.0.2
2. IDE Used: IntelliJ IDEA 2024.2.0.2 (Ultimate Edition)
3. Resources for sprites and bullet: from **dogchicken** at opengameart.org

<https://opengameart.org/users/dogchicken>

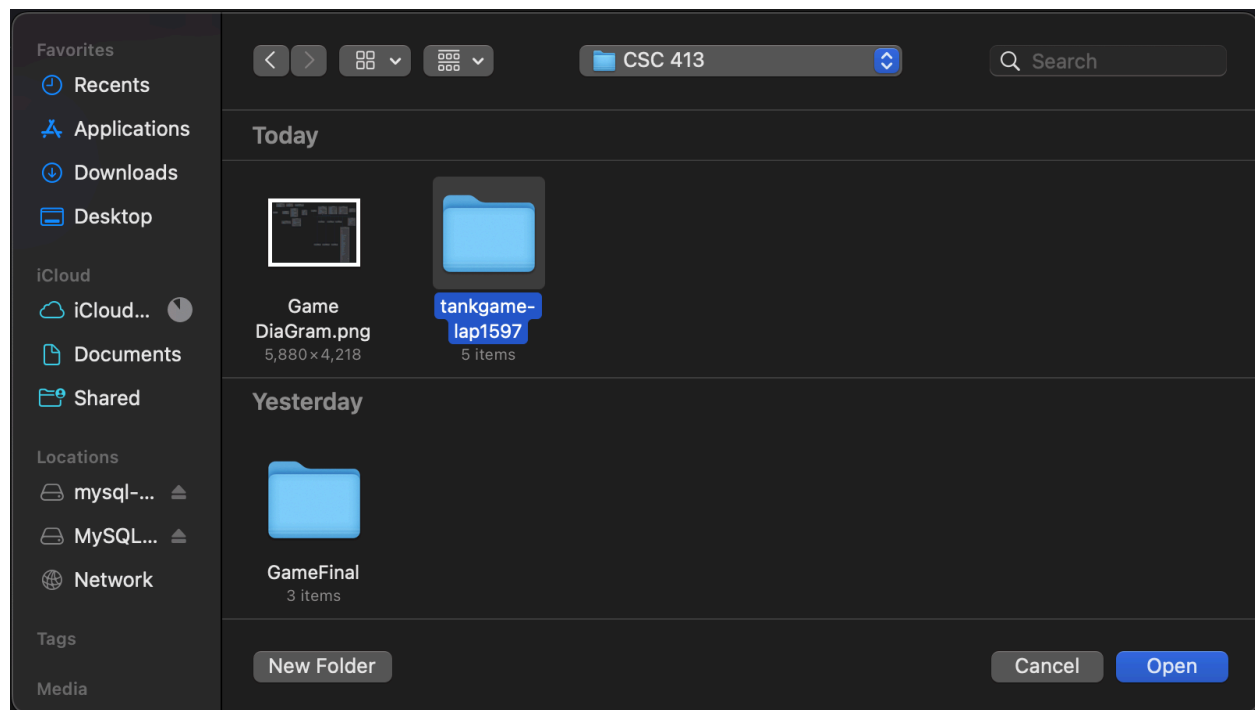
How to build

1. Importing project

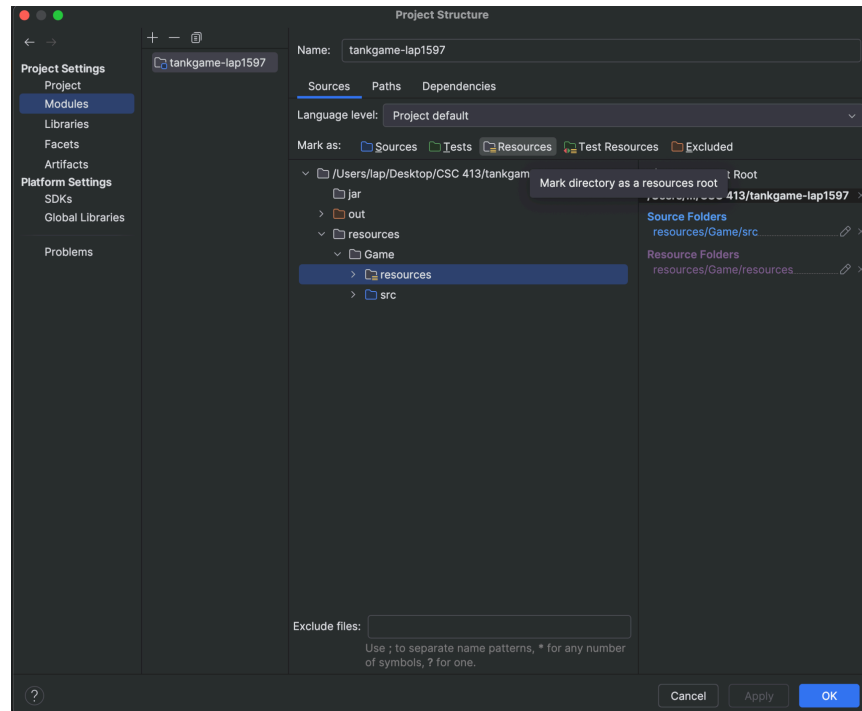
- a. Cloning the repository to the folder that you want.



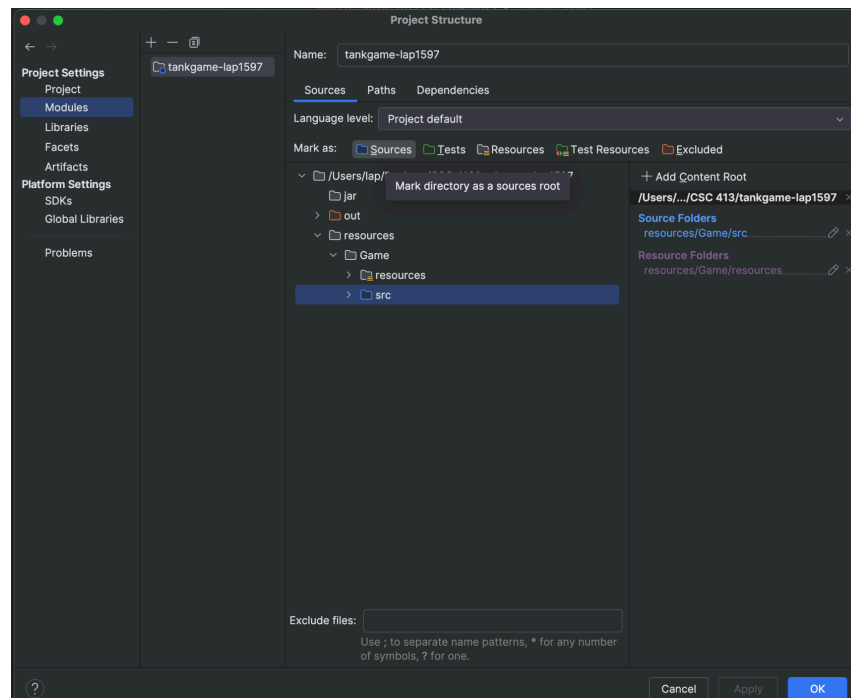
- b. Open IntelliJ -> Choose Open and find the directory that you just cloned.



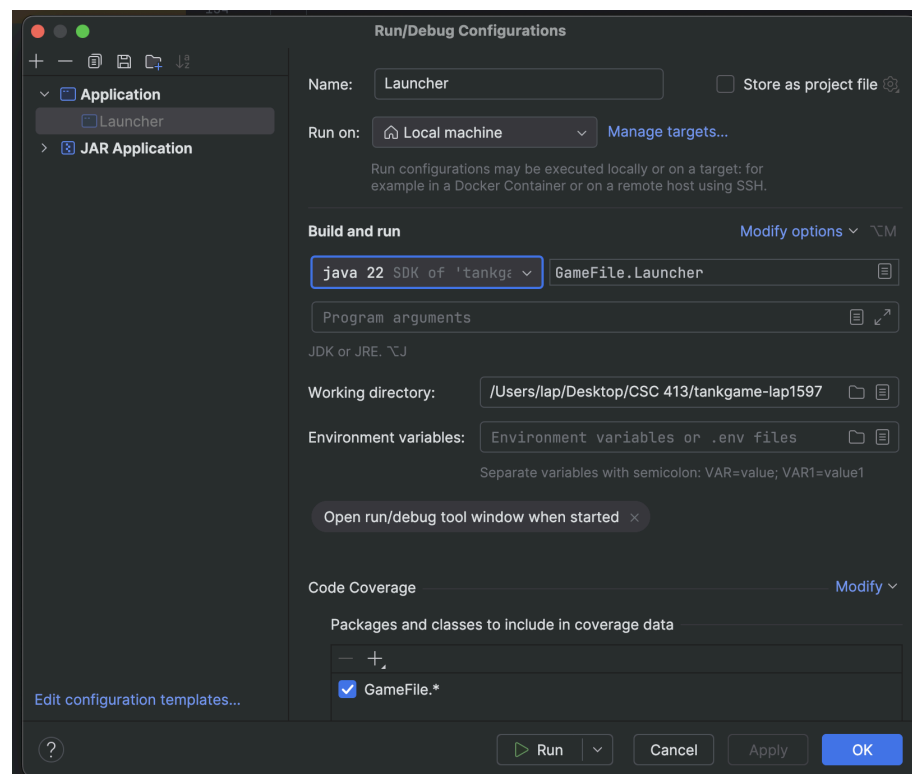
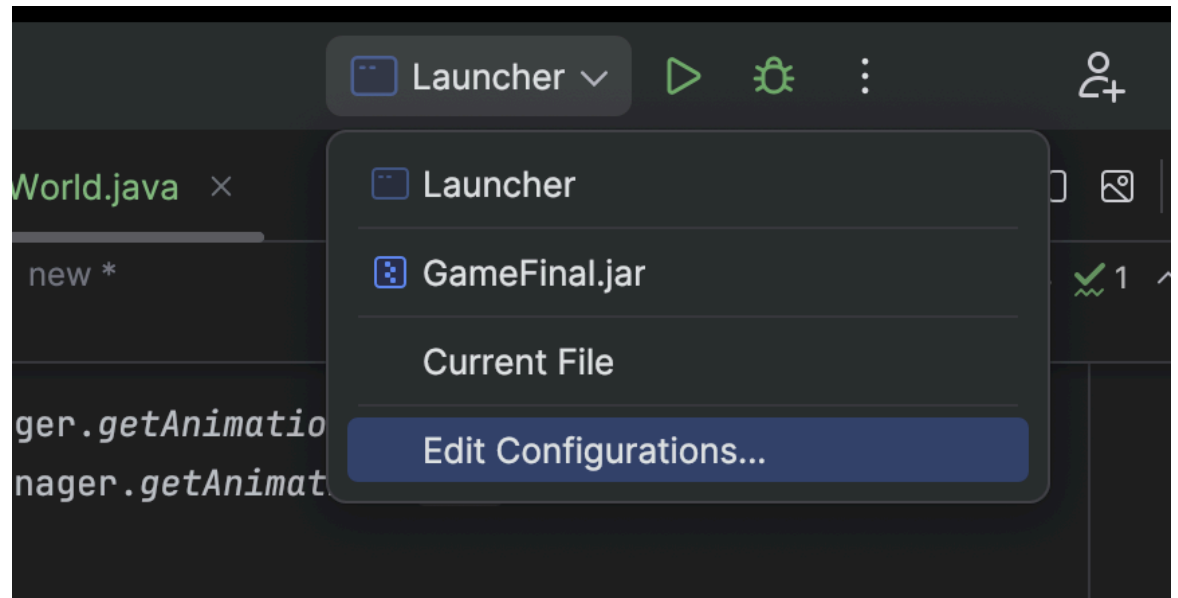
- c. Open Project Setting and set resources in Game as Resources folder.



- d. Set src as Sources



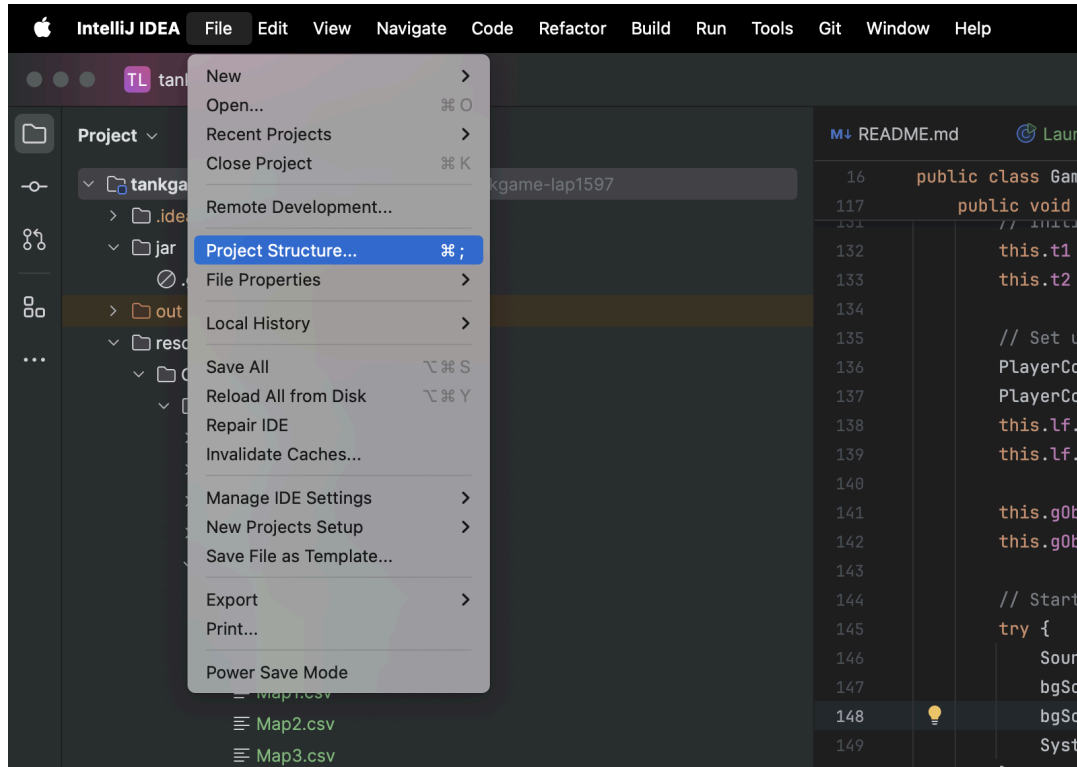
- e. Open Edit Configuration to ensure the java version.



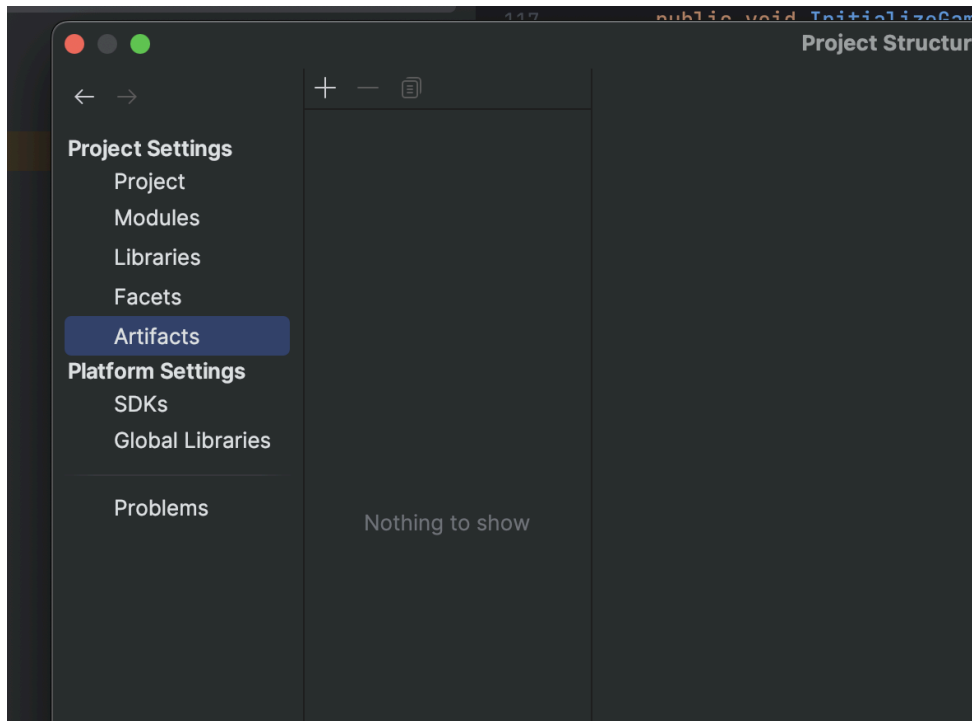
-> If Everything is match, you can start to Run to test the Game.

2. How to build JAR

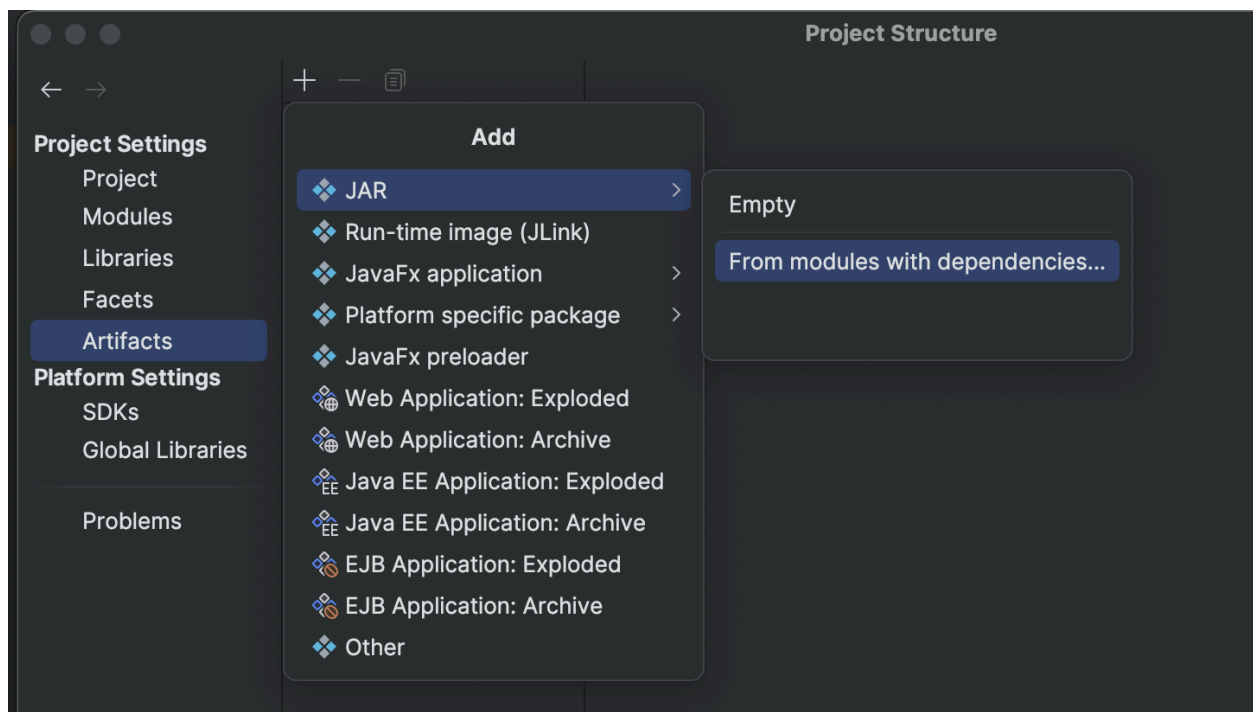
a. Open Project Structures.



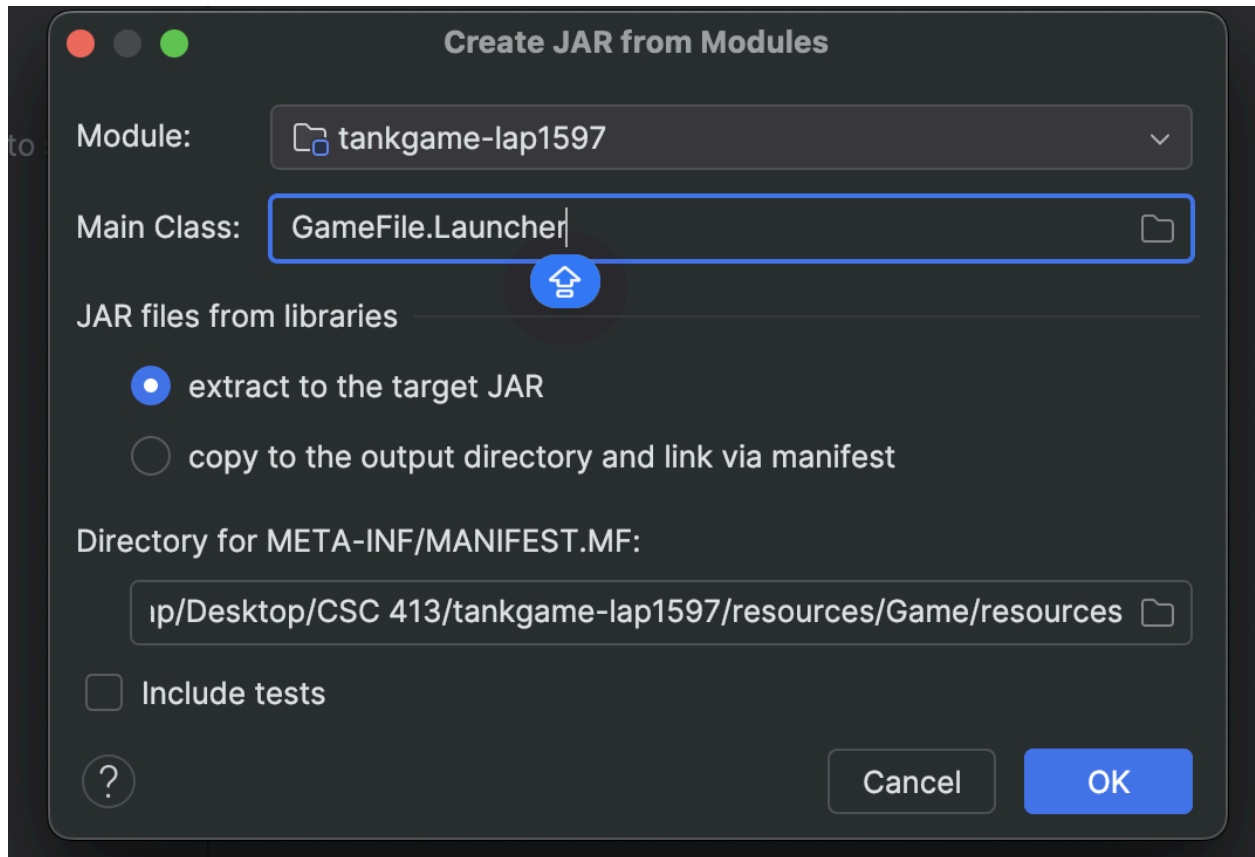
- b. Choose Artifact and Click on “+” :



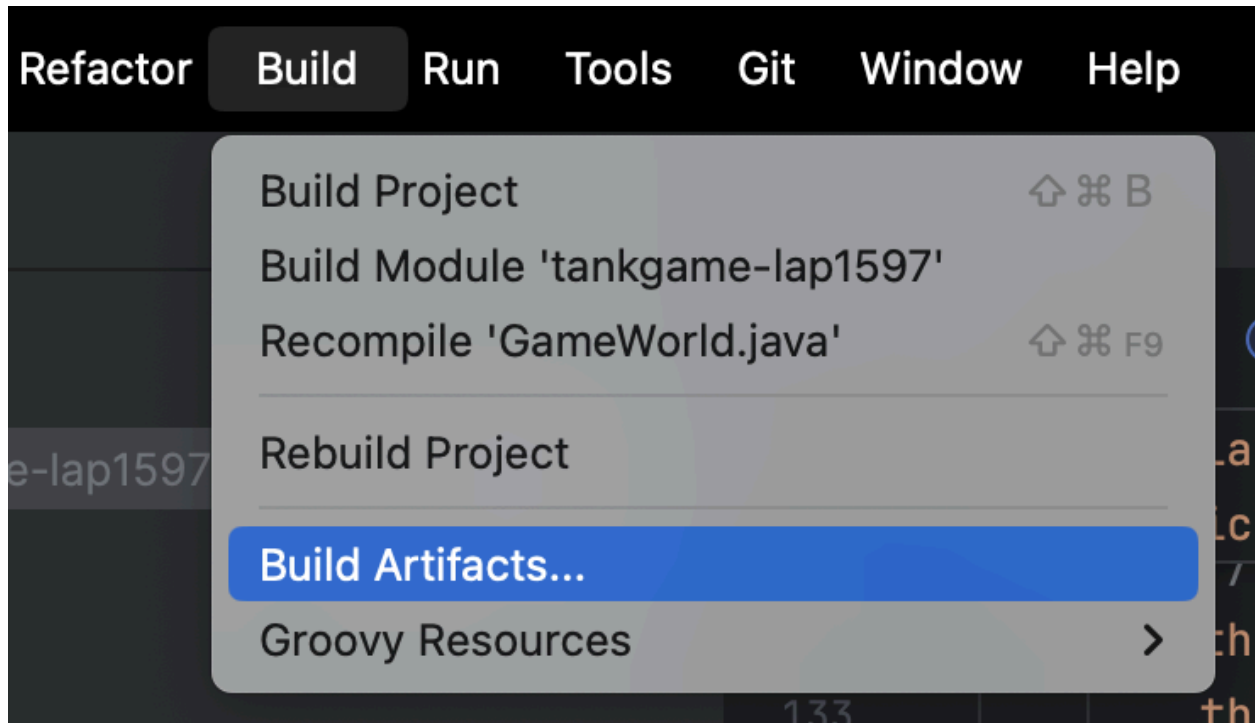
- c. Following the step in the image



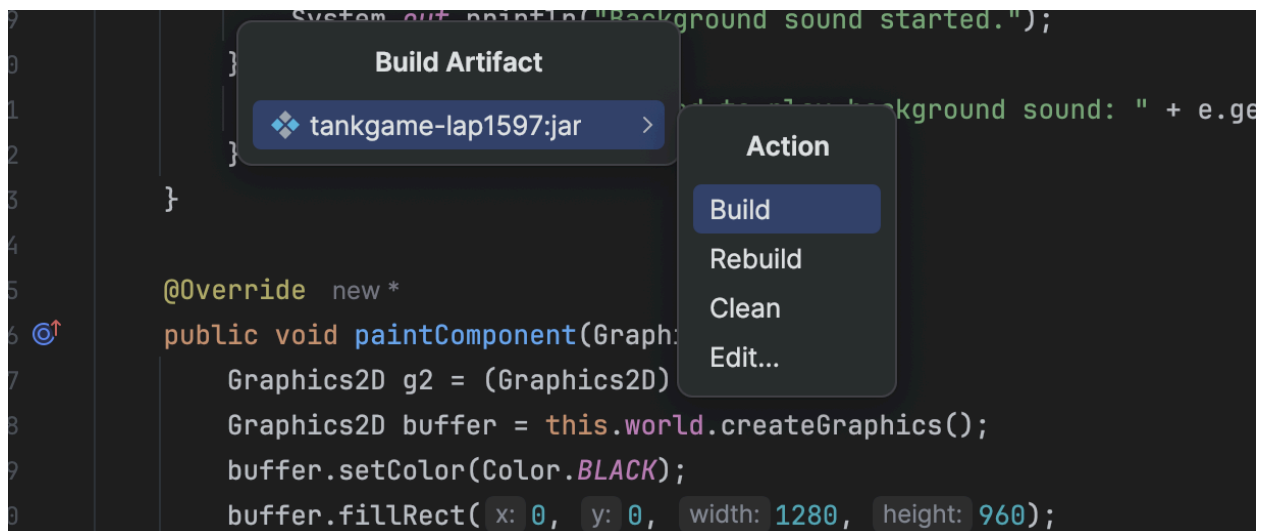
d. Set Main class as **Launcher**



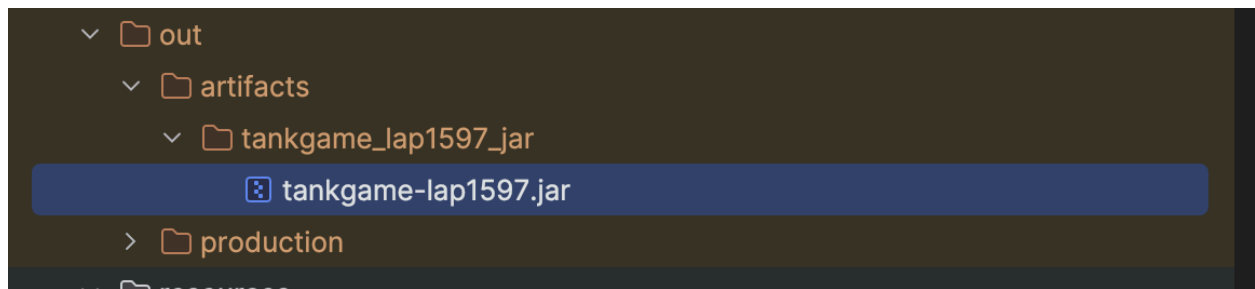
e. After completing above steps, at build section -> Choose Build Artifacts.



Then choosing Build in Action Table



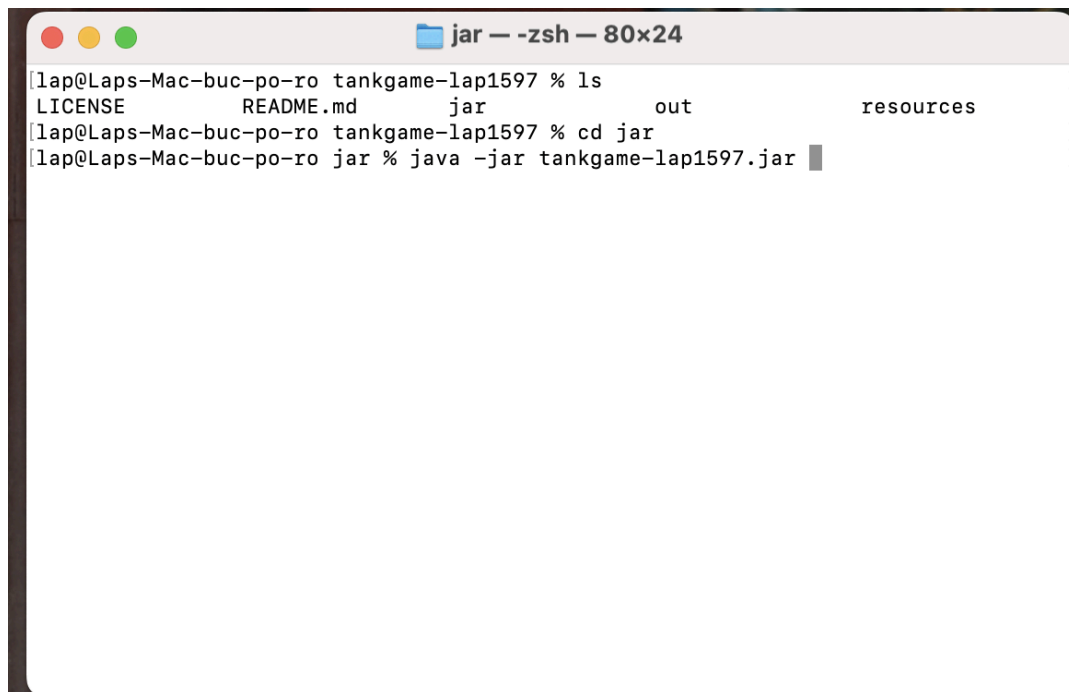
Result: You will see a jar file in the **OUT** folder.



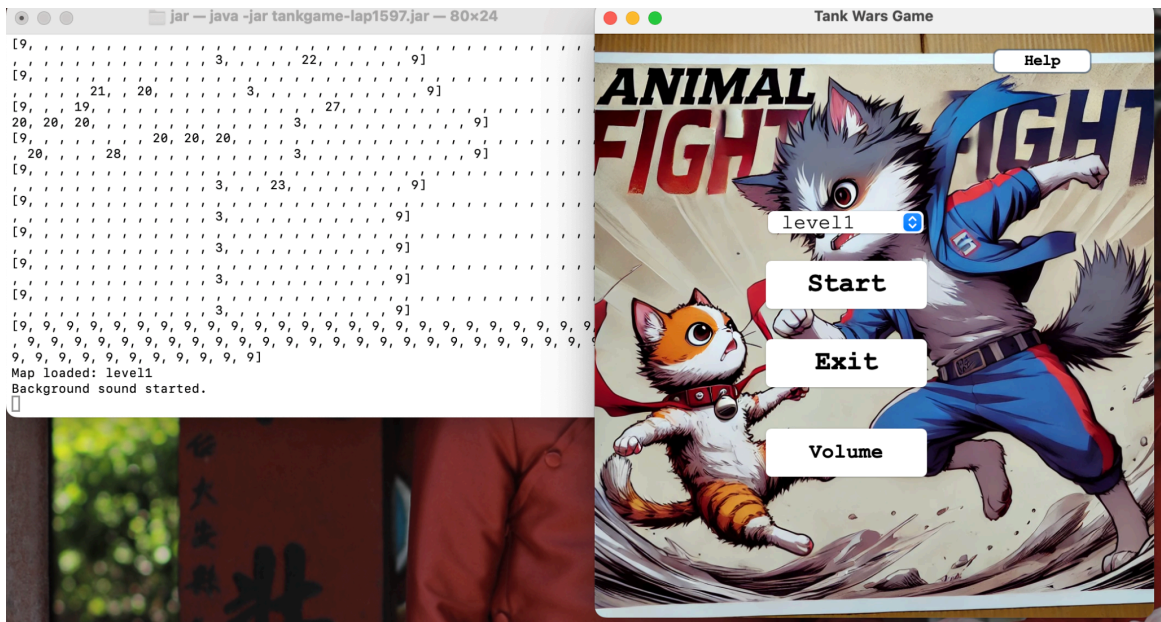
3. How to run the Game.

- a. Open terminal and redirect to where the artifacts folder located or where you store the jar file.

Using command : **java -jar [FILE'S NAME].jar**



Result:



4. Game rules:

For Dog Player:

- Use the **W**, **S**, **A**, and **D** keys to move your character Up, Down, Left, and Right.
- Press the **Spacebar** to shoot. The shooting direction is based on your movement.

For Cat Player:

- Use the arrow keys (**Up**, **Down**, **Left**, and **Right**) to move your character.
- Press **Shift** to shoot. The shooting direction is based on your movement.

General:

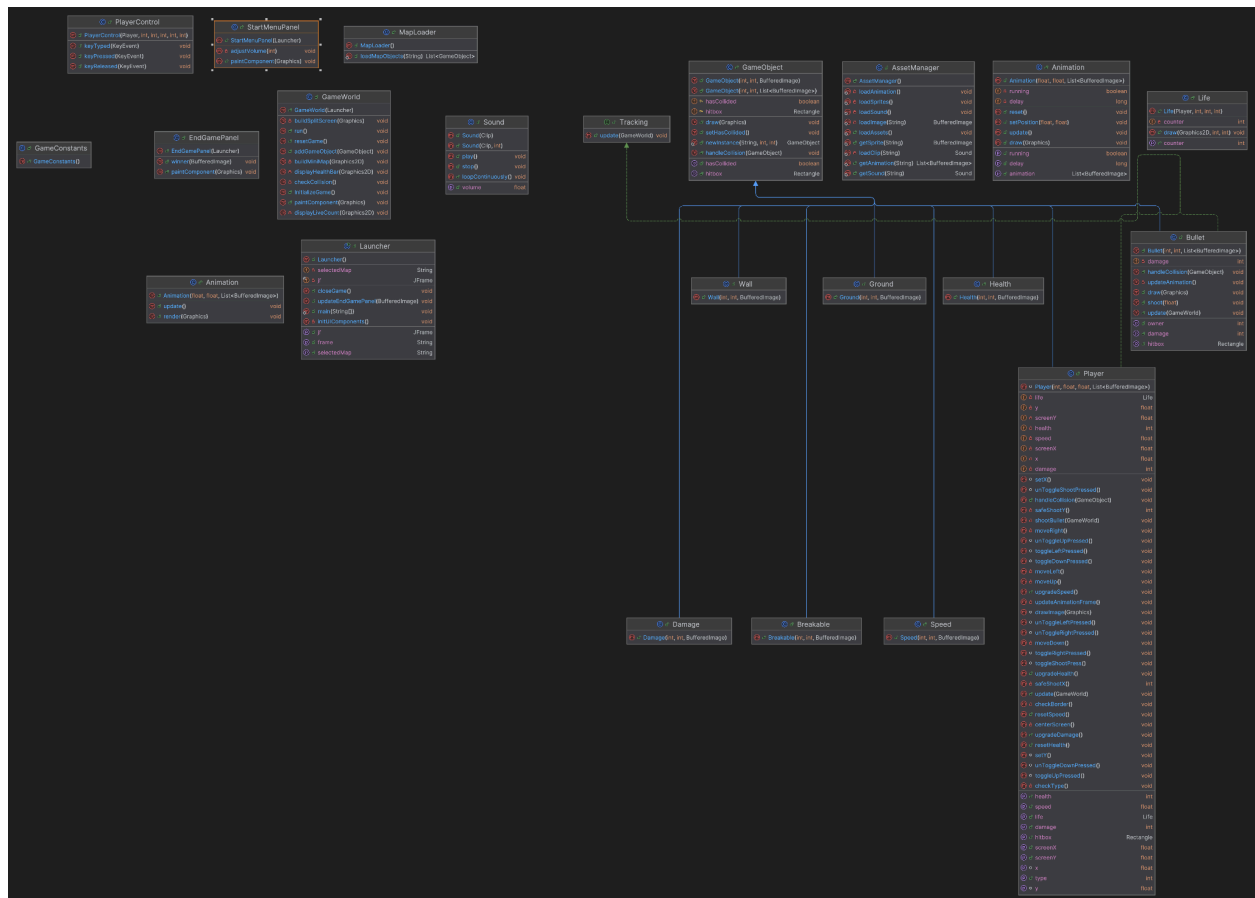
- Collect bags to increase:
 - Health - max is 100.
 - Speed - increase 0.5f each item.
 - Damage - increase 10 points each item.

Each player would have 3 lives.

5. Game Assumptions:

- No crashing while running the game.
- The collision effects trigger when it hits other objects.

6. Diagram:



7. Class Descriptions:

Game Folder:

- **Animation:** This class is responsible for animation in the game, such as explosions.
- **Breakable:** This class represents breakable walls.
- **Bullet:** This class represents the bullets that players shoot during the game and handles their collisions and the damage when it hits another player.
- **Damage:** This class manages damage items.

- **GameObject**: This abstract class serves as a base for various classes, such as **Breakable**, **Bullet**, **Player**, etc. Some functions in here will help to handle the collision.
- **GameWorld**: This class represents the game world where I start to build screens, objects and update the game status. Static and non-static entities come to life through this class.
- **Health**: This class manages health items.
- **Life**: This class represents the player's health and life count.
- **Player**: This class manages the player, including movement, shooting, and collision with other objects.
- **PlayerControl**: This class handles game controls for the player.
- **Sound**: This class manages sound effects in the game.
- **Speed**: This class manages speed items.
- **Tracking**: An interface for **Bullet** and **Player** that requires a special update mechanism.
- **Wall**: This class represents unbreakable walls.

Menu Folder:

- **StartGamePanel**: This class represents the start game menu, allowing the player to set the map, volume, and start the game.
- **EndGamePanel**: This class represents the end game menu, allowing the player to replay or exit the game.

Utils Folder:

- **AssetManager**: This class is responsible for loading game assets, such as sprite sheets, sounds, and animations.
- **MapLoader**: This class is responsible for loading maps and entities within the maps.

GameFile Folder:

- **GameConstant:** This class contains static final variables for the game such as World width, height, or Screen width, height.
- **Launcher:** This class is responsible for launching the game application by calling initComponents from GameWorld.

8. Reflection:

I initially used an external library, but due to the complexity and the limited time I had during the semester, I decided to switch back to the starter code provided by the professor. However, exploring another library to implement a game was a valuable learning experience. This project helped me gain a deeper understanding of OOP concepts and code reusability.

9. Project Conclusion:

The game runs successfully and still has many features and abilities that I can add to enhance the gameplay experience in future.